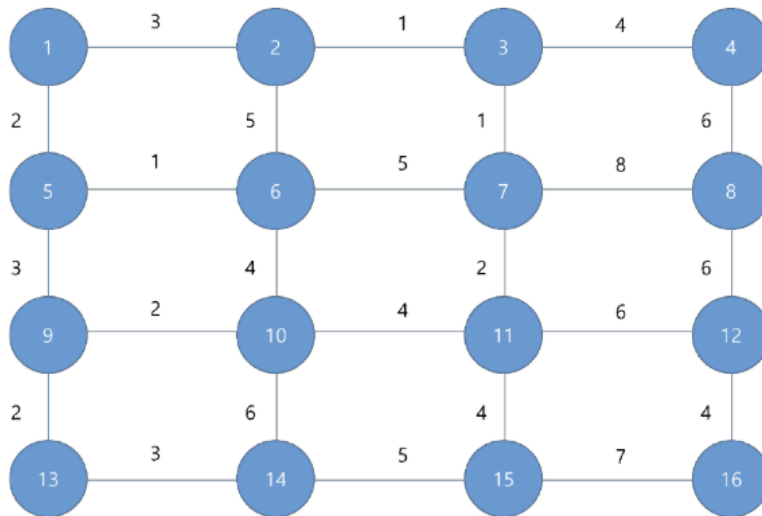


Assignment #3



- 과목명: 자료구조
- 담당교수님: 김무철 교수님
- 학과: 소프트웨어학부
- 보고서작성자: 서정민

1. 문제 상황



(1) Use the Kruskal method to find the minimum spanig tree (MST).

(2) Find the shortest path from node 1, in the graph to all nodes.

2. code

Graph를 인접 리스트 형태로 main 함수에 input하여 크루스칼 알고리즘에 사용하여 MST를 구했고, 함수를 통해 인접 행렬 형태로 변환하여 문제 2에서 다익스트라 알고리즘(음수 weight가 없으므로)에 사용하였다.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <limits.h>
4
5  #define V 16
6  #define E 24
```

이 문제에서 vertex는 16개, edge는 24개.

```
8  //특정 원소가 속한 집합을 찾는 find함수
9  int find(int parent[], int i) {
10     if (parent[i] != i)
11         parent[i] = find(parent, parent[i]);
12     return parent[i]; //재귀적으로 parent를 찾아가 최상위 노드를 찾고 반환
13 }
```

크루스칼 알고리즘에서 간선을 선택하는 과정에서, 간선을 추가시 사이클이 생기는지 확인해야 하고, 사이클이 생기지 않는 경우에만 간선을 MST에 추가한다.

find 함수를 통해서 재귀적으로 parent를 찾아가서 최상위 노드를 찾고 반환하여 집합을 확인한다.

```
15 //두 집합 합치는 Union
16 // 사실 어느쪽에 합치든 상관없으나 유니온 파인드 자료구조에서는 랭크를 기준으로 트리를 병합한다
17 // 트리의 높이를 줄일 수 있고, find 연산의 평균 시간복잡도를 줄일 수 있다.
18 void Union(int parent[], int rank[], int x, int y) {
19     int root1 = find(parent, x);
20     int root2 = find(parent, y);
21
22     if (rank[root1] < rank[root2]) {
23         parent[root1] = root2;
24     }
25     else if (rank[root1] > rank[root2]) {
26         parent[root2] = root1;
27     }
28     else {
29         parent[root2] = root1;
30         rank[root1]++;
31     }
32 }
```

크루스칼 알고리즘에 사용하는 유니온 파인드 자료구조는 랭크를 기준으로 트리를 병합한다.

어느쪽에 합쳐도 결과는 나오게 되지만 랭크를 기준으로 트리를 병합하여 트리의 높이를 줄여서 find 함수의 평균 시간복잡도를 줄일 수 있다.

```
35 //compare - qsort
36 int compare(const void* a, const void* b) {
37     int* edgeA = (int*)a;
38     int* edgeB = (int*)b;
39     if (edgeA[2] > edgeB[2])
40         return 1;
41     else if (edgeA[2] < edgeB[2])
42         return -1;
43     else
44         return 0;
45 }
```

qsort에 사용되는 compare 함수. 크루스칼 알고리즘에는 간선을 가중치순으로 정렬하고 가중치가 작은 간선부터 선택하므로 정렬 과정이 필요하다.

```

47 // 1. kruskal_MST
48 /*
49 간선을 가중치 기준 정렬
50 가중치가 가장 작은 간선부터 선택
51 유니온파인드 자료구조를 이용하여 간선 추가시 사이클이 생기나 확인
52 사이클이 안 생기는경우에 간선을 MST에 추가, MST 완성될때까지 반복
53 */
54 void kruskal_MST(int input_graph_list[E][3]) {
55     int edges[E][3];
56     for (int i = 0; i < E; i++) {
57         edges[i][0] = input_graph_list[i][0] - 1;
58         edges[i][1] = input_graph_list[i][1] - 1;
59         edges[i][2] = input_graph_list[i][2];
60     }
61
62     qsort(edges, E, sizeof(edges[0]), compare);
63
64     int parent[V]; // parent는 초기에 자신을 부모로 갖도록 초기화됨
65     int rank[V];
66     for (int v = 0; v < V; v++) {
67         parent[v] = v;
68         rank[v] = 0;
69     }

```

```

71 int MST[V - 1][3]; //MST에는 V-1개의 간선이 포함된다
72 int e = 0; //mst의 간선 수
73 int i = 0; //정렬된 간선 배열의 인덱스
74
75 while (e < V-1 && i < E){ // mst는 V-1개의 간선을 가져야 하고, 모든 간선을 처리할때까지 반복
76     int* edge = edges[i]; // 이미 배열이 가중치 기준으로 정렬되어 있으므로, 작은 가중치 간선부터 선택하여 처리
77     i++;
78     int x = find(parent, edge[0]); // x,y는 루트 노드
79     int y = find(parent, edge[1]);
80
81     if (x != y) { //루트노드가 다른 경우 간선을 mst에 추가해도 사이클이 생기지 않는다
82         MST[e][0] = edge[0]; // 간선을 mst에 추가
83         MST[e][1] = edge[1];
84         MST[e][2] = edge[2];
85         e++;
86         Union(parent, rank, x, y); //union함수를 사용하여 두 집합을 합친다
87     }
88
89 }
90 for (i = 0; i < e; i++)
91     printf("[%d] - [%d], weight:%d \n", MST[i][0] + 1, MST[i][1] + 1, MST[i][2]);
92
93 }

```

크루스칼 알고리즘으로 MST를 찾는다. 처음 main 함수에 input한 인접리스트를 사용했다.

간선을 가중치 기준으로 정렬하고, 가중치가 가장 작은 간선부터 선택한다. 간선이 추가시 사이클이 생기는지 확인하면서 사이클이 생기지 않는 경우에만 간선을 MST에 추가한다. 이 과정을 MST가 완성될때까지 반복한다.

```

95 //2. dijkstra - Shortest Path from Node 1
96 void dijkstra(int graph_matrix[V][V], int src) {
97     int d[V]; // 출발점에서 각 정점까지의 최단 거리 배열, 초기값 무한대
98     int p[V]; // 각 정점의 이전 정점 배열, 초기값 -1
99     int sptSet[V]; // 최단 경로 트리에 포함된 정점 집합
100
101     for (int i = 0; i < V; i++) {
102         d[i] = INT_MAX;
103         p[i] = -1;
104         sptSet[i] = 0;
105     }

```

```

107 d[src] = 0; // 출발점의 거리는 0
108
109 //모든 정점에 대해서 최단경로 찾기
110 for (int count = 0; count < V - 1; count++) {
111     int min = INT_MAX, min_index;
112     for (int v = 0; v < V; v++)
113         if (sptSet[v] == 0 && d[v] < min) {
114             // 아직 최단 경로 트리에 포함되지 않은 정점 중에서 최단 거리를 가지는 정점 선택
115             min = d[v];
116             min_index = v;
117         }
118     int u = min_index; //최소 거리 정점
119     sptSet[u] = 1; //선택된 정점을 최단 경로 트리에 포함
120
121     //선택된 정점의 인접 정점의 거리 값 갱신
122     for (int v = 0; v < V; v++)
123         if (!sptSet[v] && graph_matrix[u][v] && d[u] != INT_MAX && d[u] + graph_matrix[u][v] < d[v]) {
124             d[v] = d[u] + graph_matrix[u][v];
125             p[v] = u; // 이전 정점을 현재 정점으로 설정
126         }
127 }
128 printf("vertex\tdistance\tpath\n");
129 for (int i = 0; i < V; i++) {
130     printf("%d \t %d\t\t", i + 1, d[i]);
131     // 경로 출력
132     int path[V];
133     int count = 0;
134     for (int v = i; v != -1; v = p[v])
135         path[count++] = v;
136     for (int j = count - 1; j >= 0; j--)
137         printf("%d ", path[j] + 1);
138     printf("\n");
139 }
140 }

```

다익스트라 알고리즘을 통해서 shortest path from node 1을 구한다. 처음 main함수에 input한 인접리스트를 인접행렬로 바꾼 것을 사용했다.

d배열은 출발점에서 각 정점까지의 최단거리 배열이고, 초기값은 무한대로 한다.

p배열은 각 정점의 이전 정점 배열이고, 초기값은 -1로 한다

sptSet배열은 최단 경로 트리에 포함된 정점 집합이고, 0으로 초기화 한다.

모든 정점에 대해서 최단경로를 찾는 과정으로, 아직 최단 경로 트리에 포함되지 않은 정점 중에서 최단거리를 갖는 정점을 선택하고, 선택된 정점을 최단 경로 트리에 포함한다. 그리고 선택된 정점의 인접 정점의 거리 값을 갱신하고, 이전 정점을 현재 정점으로 설정한다.

출력 과정에서는 path 배열을 이용하여 경로 전체가 출력되도록 한다.

```

143 //input받은 인접 리스트를 인접 행렬로 변환하여 사용하기
144 void convert_list_to_matrix(int input_graph_list[E][3], int graph_matrix[V][V]) {
145     for (int i = 0; i < V; i++) {
146         for (int j = 0; j < V; j++) {
147             graph_matrix[i][j] = 0;
148         }
149     }
150
151     for (int i = 0; i < E; i++) {
152         int node1 = input_graph_list[i][0] - 1;
153         int node2 = input_graph_list[i][1] - 1;
154         int weight = input_graph_list[i][2];
155
156         graph_matrix[node1][node2] = weight;
157         graph_matrix[node2][node1] = weight;
158     }
159 }
160

```

인접 리스트를 인접 행렬로 변환하는 함수. 기본값을 0으로 초기화하고, 이 그래프에서 행렬은 대칭이 된다.

```

162 // 인접행렬 출력함수
163 void printMatrix(int graph_matrix[V][V]) {
164     for (int i = 0; i < V; i++) {
165         for (int j = 0; j < V; j++) {
166             printf("%d ", graph_matrix[i][j]);
167         }
168         printf("\n");
169     }
170 }

```

인접행렬 출력함수

```

172  int main() {
173      // node1,node2,weight 순서, 총 24개 edge
174      int input_graph_list[E][3] = {
175          {1,2,3},{2,3,1},{3,4,4},
176          {1,5,2},{2,6,5},{3,7,1},{4,8,6},
177          {5,6,1},{6,7,5},{7,8,8},
178          {5,9,3},{6,10,4},{7,11,2},{8,12,6},
179          {9,10,2},{10,11,4},{11,12,6},
180          {9,13,2},{10,14,6},{11,15,4},{12,16,4},
181          {13,14,3},{14,15,5},{15,16,7}
182      };
183
184      int graph_matrix[V][V];
185      convert_list_to_matrix(input_graph_list,graph_matrix);
186
187      printMatrix(graph_matrix);
188
189
190      printf("\n");
191      printf("1. Kruskal - MST\n");
192      kruskal_MST(input_graph_list); // 인접리스트를 사용한다
193
194
195      printf("\n");
196      printf("2. dijkstra - Shortest Path from Node 1\n");
197      dijkstra(graph_matrix, 0); // 인접 배열을 사용한다
198
199      getch();
200      return 0;
201  }

```

main함수에서 주어진 그래프를 인접 리스트 형태로 입력했고, convert_list_to_matrix함수를 통해서 인접행렬로 변환하여 사용하였다.

3. 결과

```

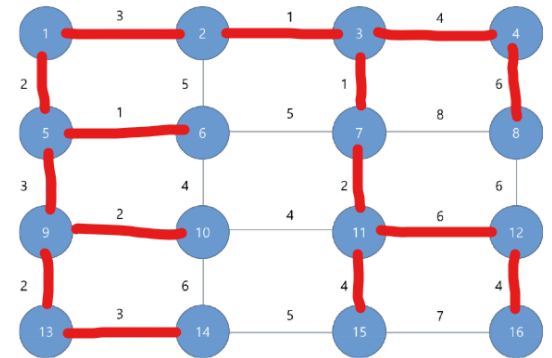
0 3 0 0 2 0 0 0 0 0 0 0 0 0 0 0
3 0 1 0 0 5 0 0 0 0 0 0 0 0 0 0
0 1 0 4 0 0 1 0 0 0 0 0 0 0 0 0
0 0 4 0 0 0 0 6 0 0 0 0 0 0 0 0
2 0 0 0 0 1 0 0 3 0 0 0 0 0 0 0
0 5 0 0 1 0 5 0 0 4 0 0 0 0 0 0
0 0 1 0 0 5 0 8 0 0 2 0 0 0 0 0
0 0 0 6 0 0 8 0 0 0 0 6 0 0 0 0
0 0 0 0 3 0 0 0 0 2 0 0 2 0 0 0
0 0 0 0 0 4 0 0 2 0 4 0 0 6 0 0
0 0 0 0 0 0 2 0 0 4 0 6 0 0 4 0
0 0 0 0 0 0 0 6 0 0 6 0 0 0 0 4
0 0 0 0 0 0 0 0 2 0 0 0 0 3 0 0
0 0 0 0 0 0 0 0 0 6 0 0 3 0 5 0
0 0 0 0 0 0 0 0 0 0 4 0 0 5 0 7
0 0 0 0 0 0 0 0 0 0 0 4 0 0 7 0

```

인접리스트를 인접 행렬로 바꾼 결과물

```
1. Kruskal - MST
[2] - [3], weight:1
[5] - [6], weight:1
[3] - [7], weight:1
[9] - [13], weight:2
[1] - [5], weight:2
[9] - [10], weight:2
[7] - [11], weight:2
[13] - [14], weight:3
[1] - [2], weight:3
[5] - [9], weight:3
[3] - [4], weight:4
[12] - [16], weight:4
[11] - [15], weight:4
[11] - [12], weight:6
[4] - [8], weight:6
```

(1) Use the Kruskal method to find the minimum spanig tree (MST). 문제를 해결한 결과



2. dijkstra - Shortest Path from Node 1		
vertex	distance	path
1	0	1
2	3	1 2
3	4	1 2 3
4	8	1 2 3 4
5	2	1 5
6	3	1 5 6
7	5	1 2 3 7
8	13	1 2 3 7 8
9	5	1 5 9
10	7	1 5 6 10
11	7	1 2 3 7 11
12	13	1 2 3 7 11 12
13	7	1 5 9 13
14	10	1 5 9 13 14
15	11	1 2 3 7 11 15
16	17	1 2 3 7 11 12 16

(2) Find the shortest path from node 1, in the graph to all nodes 문제를 해결한 결과